

ask-gen3

Grounded question answering over the Gen3 codebase

27 repositories · 15,749 chunks · one 55 MB file · \$0.0023 per answer

The problem

Gen3 is a data-commons platform spread across **27 public repositories**.

Answering "*how does fence issue a refresh token?*" means:

- knowing it's `fence`, not `arborist` or `wt`
- finding `oidc/grants/refresh_token_grant.py`
- reading the code, not the README's summary of it

Newcomers can't do step 1. Search engines don't index step 2 well.

What it does

In: a natural-language question.

Out: a prose answer where every claim carries a citation to real source.

```
[fence/fence/oidc/grants/refresh_token_grant.py#L34-L71]
```

The citation is a file, a path, and **line numbers that resolve**.

The one promise

Never answer from memory. Every claim comes from a tool result.

An LLM that has read Gen3 during pretraining will happily invent `fence/auth/refresh.py`. It sounds right. It does not exist.

The entire design exists to make that failure impossible.

Big picture: two phases

BUILD (offline, ~1 hour, free)

clone 27 repos

↓

chunk into 15,749 pieces

↓

embed each chunk

↓

write index.db (55 MB) →

QUERY (online, \$0.002 typical)

question

↓

embed it (same model)

↓

search index.db

↓

feed chunks to an LLM

↓

cited answer

Why split them?

ADR-0005: the index is a build artifact.

	Build	Query
Runs	on demand, manually	per request
Takes	~1 hour	seconds
Needs	git, network, 27 clones	one file
Costs	nothing	~\$0.002

The server never clones anything. It opens a file.

Build: chunking

Not fixed-size windows. Chunks follow **structure**:

- **prose** — markdown sections, split on headings
- **code** — functions and classes, via language-aware parsing
- **openapi** — per-endpoint
- **schema** — per-definition

Each chunk records `repo`, `path`, `start_line`, `end_line`, `symbol`, `heading`.

Build: embeddings

`BAAI/bge-small-en-v1.5` , running as **ONNX on the local CPU**.

- 384 dimensions
- 64 MB quantized model file
- 15,749 chunks embedded in one pass
- **cost: \$0.00**

ADR-0002 rejected hosted embedding APIs. At corpus scale that's a real bill; here it's a laptop running for an hour.

The artifact

One SQLite file. 55 MB. Three roles in one:

Table	Holds	Serves
chunks	text + repo/path/lines	content and citations
chunks_fts	FTS5 inverted index	keyword (BM25)
chunks_vec	15,749 × 384 floats	semantic

ADR-0001: SQLite is the entire retrieval layer. No Elasticsearch, no pgvector, no vector database, no running service.

Query: hybrid retrieval

Two searches, always. They fail in **opposite directions**.

	Good at	Bad at
BM25	exact identifiers: <code>ssjdispatcher</code> , <code>presigned_url</code>	paraphrase, concepts
Vectors	"how does auth work"	rare tokens never seen in training

`ssjdispatcher` was not in bge-small's training data. Its vector is close to noise. BM25 doesn't care — exact match is its *best* case.

The tokenizer detail that matters

```
tokenize="unicode61 tokenchars '_.-/'"
```

Input	Default FTS5	Ours
refresh_token_grant.py	refresh token grant py	one token
fence/jwt/token.py	fence jwt token py	one token

Default splitting means searching token.py matches every chunk containing "token" — thousands, all useless.

One line of config, most of the lexical half's value.

Fusing the two

Reciprocal rank fusion — combine by **rank**, not score.

BM25 scores and cosine similarities are not comparable quantities. Normalising them is a tuning exercise with no principled answer.

Ranks are always comparable. RRF needs no weights, no calibration, and no retuning when the corpus grows.

Query: an agent, not a pipeline

Classic RAG: retrieve once, stuff context, generate. **One shot.**

Here the model gets **five tools** and decides:

Tool	For
search	concepts, hybrid retrieval
grep	exact identifiers
open_file	read more around a hit
list_repos	orient — what's even indexed
fetch_url	read a page the index doesn't hold

Why an agent helps here

"Walk me through a controlled-access file download."

Single-shot retrieval gets chunks about one service.

The loop does: `search` fence → `grep` presigned → `open_file` the handler → `search` arborist → answer spanning **three services**.

Cross-service questions are where the codebase is hardest to learn, and exactly where one-shot RAG fails.

Reaching outside the index

Release notes and the docs site aren't in the corpus, and the index is a snapshot besides. `fetch_url` is the one tool that leaves the process.

ADR-0009: a built-in fetch tool, not the reference MCP server. The question was never who fetches the bytes — it's that **the model picks the URL**. On Cloud Run the first useful target is `169.254.169.254`.

- https only, four allowlisted hosts, `uc-cdis` as the first path segment on GitHub
- redirects never followed blindly — every hop re-checked
- every resolved address must be public
- results banner-marked untrusted, cited as URLs, not as `[repo/path#L]`

No framework

ADR-0004: hand-rolled loop, no LangChain / LlamaIndex.

The whole agent is one readable file.

- every token sent to the model is visible in one place
- prompt changes don't fight a framework's templating
- no dependency that rewrites its API each minor version

The loop is roughly: call model → if tool calls, run them → append results → repeat. That's genuinely all it is.

Follow-ups, without a session store

The page keeps the transcript and posts it back with every question. **The server stores nothing** — which is what lets any instance serve any turn.

- No turn limit. History fills a **6 k token budget, newest first** — a long exchange of short questions survives whole; one verbose answer gets truncated rather than evicting four others.
- Only text travels. Prior tool results are dropped, because history is re-sent on every call — replaying them would spend the next question's budget.
- History arrives in a caller-controlled body, so only `user` and `assistant` strings are admitted. **A forged `system` turn never reaches the model.**

The ceiling is a budget, not a step count

ADR-0010. The loop used to stop at six tool calls. Six cheap rounds and six expensive ones differ by more than an order of magnitude and hit the same ceiling.

Every call re-sends the whole conversation, so the bill is the **sum** of the prompts, not the size of the largest one.

	Before	Now
Ceiling	6 tool calls, ~25 k prompt tokens	180 k tokens across the question
Meaning	a proxy for cost	~USD 0.05, the real number
Cheap question	capped at 6	as many rounds as it needs

Out of budget → the next call is offered no tools, so it has to answer.

Grounding, enforced

Four mechanisms, not just a hopeful prompt:

1. **Tool results only.** Model is told to never answer from memory.
2. **Citations must come from tool output** — markers are emitted by retrieval, not composed by the model.
3. **Repo and web content is untrusted.** A file containing "ignore your instructions" gets *reported*, not obeyed.
4. **Hard ceilings** — 180 k billed tokens and ~USD 0.05 per question, 2,000-char questions, 120 s wall clock.

Choosing the model: measured, not guessed

ADR-0007 benchmarked 8 candidates.

Model	Usable	Valid cites	Fabricated	\$/answer
gemini-3.1-flash-lite	3/3	20	0	0.0073
gemini-2.5-flash-lite	3/6	8	0	0.0026
gpt-5-nano	1/3	1	0	0.0016
mistral-nemo (<i>incumbent</i>)	0/3	0	2	0.0004

The cheapest model invented file paths — the one failure this app exists to prevent. **Quality spread (0/3 → 3/3) far exceeds the 17× price spread.**

Deployment: three attempts

Attempt	Outcome
AWS Lambda	Function URL returned 403 to <i>everything</i> . Correct policy, correct auth type, direct invoke fine, no CloudWatch entry. Account-level block.
HF Spaces	<code>Quota exceeded for cpu-basic: limit=0</code>
Cloud Run	Live

Why Cloud Run fits

- **Scales to zero** — idle costs nothing, and traffic is bursty
- **Streams SSE natively** — no adapter, unlike Lambda
- **Container, unmodified** — same image runs under `docker run`
- 1 vCPU / 1 GiB / max 3 instances

The 55 MB index ships **inside the image**. No volume, no bucket, no cold-start download.

Cost: per answer

Component	Per answer	Share
OpenRouter inference	\$0.0020	87%
Cloud Run CPU	\$0.00026	11%
Cloud Run memory	\$0.00003	1%
Requests, egress	~\$0.000001	~0%

Inference dominates. Everything else is noise.

Latency costs twice: the container is billed while blocked on the API.

Cost: fixed monthly

Item	Cost
Secret Manager (1 version)	\$0.06
Artifact Registry (596 MB)	\$0.01
Cloud Build, Logging	\$0.00 (free tier)
Idle compute	\$0.00 (scale to zero)
Total	~\$0.07/mo

No database, no VPC, no load balancer, no NAT.
Those are where hobby projects bleed. There are none.

Cost: bounded

Volume/mo	Total
100	\$0.30
1,000	\$2.36
10,000	\$23

Ceilings: **USD 0.05 per question** (ADR-0010) · 20 questions/IP/hour · 2,000/day/instance · max 3 instances.

The real backstop is the spend limit on the OpenRouter key — no amount of concurrency can exceed it.

CI/CD: no keys anywhere

```
permissions:
  id-token: write

- uses: google-github-actions/auth@v2
  with:
    workload_identity_provider: ${ vars.GCP_WIF_PROVIDER }
```

Workload Identity Federation: GitHub mints a short-lived OIDC token, GCP verifies and grants ~1 hour of access.

Pinned to repo *and* branch — a token from any other branch is rejected before it reaches the service account.

Pipelines

```
index (manual) → 60–180 min → index.db artifact
                    |
                    ▼
deploy (auto) → Cloud Build → Cloud Run → /health check
ci (per push) → ruff + 6 test modules → seconds
```

Index is manual by choice: the corpus moves slowly, a stale index is not an outage, and scheduled workflows auto-disable after 60 idle days.

Evals

`evals/questions.jsonl` — 50 questions with checkable expectations.

```
{"id": "fence-006",  
  "question": "How does fence generate a presigned URL?",  
  "must_include": ["indexd", "presigned", "authz"],  
  "should_cite": ["uc-cdis/fence/fence/blueprints/data/indexd.py"]}
```

Not prose answers — no scorer can match those. Facts and citations.

Includes 4 **refusal** cases: off-topic, unsafe, secret-fishing. `evals/check.py` verifies all 98 citations resolve against the index.

Honest limits

- **Snapshot, not live.** Index reflects the last manual build.
- **Seconds per answer, and a hard question is slower now.** Removing the step ceiling means ~12 rounds where it ran 6 — the 120 s timeout is what bounds it.
- **Retrieval ceiling.** A wrong chunk means a wrong answer, confidently cited to a real file.
- **The token estimator is ~2× conservative**, so the loop answers earlier than it strictly must. Safe direction, but it's a calibration knob, not a fact.
- **Per-instance rate limits.** Counters don't coordinate across the three instances; real ceilings are ~3×.
- **English only.** Embedding model is English-trained.

What it demonstrates

Boring infrastructure, carefully chosen.

- SQLite instead of a vector database
- Local ONNX instead of an embeddings API
- A hand-written loop instead of a framework
- An allowlist in the same file as the request, instead of an MCP server
- A cost ceiling instead of a step count
- Measured model choice instead of vibes
- Federated identity instead of stored keys

\$0.07/month fixed. Scales to zero. Every claim cited.

Questions?

`github.com/haraprasadj/ask-gen3`

Design decisions: `docs/adr/0001 – 0010`